Q1: Explain all operators in details.

Answer:

Operators are symbols that tell the compiler to perform specific mathematical or logical manipulations. In Object-Oriented Programming (C++), operators can be grouped into several categories based on their functionality.

1. Arithmetic Operators These perform common mathematical operations.

- + (Addition), - (Subtraction), * (Multiplication), / (Division).
- % (Modulus): Returns the remainder of an integer division.

2. Relational Operators These compare two values and return a boolean result (true or false).

- == (Equal to), != (Not equal to).
- > (Greater than), < (Less than).
- >= (Greater than or equal to), <= (Less than or equal to).

3. Logical Operators Used to combine conditional statements.

- && (Logical AND): Returns true if both statements are true.
- || (Logical OR): Returns true if at least one statement is true.
- ! (Logical NOT): Reverses the result (true becomes false).

4. Assignment Operators Used to assign values to variables.

- = (Simple assignment).
- Compound assignments: +=, -=, *=, /= (e.g., x += 5 is same as x = x + 5).

5. Increment and Decrement Operators (Unary)

- ++ (Increment): Increases integer value by 1. Can be Pre-increment (++i) or Post-increment (i++).
- -- (Decrement): Decreases integer value by 1.

6. Bitwise Operators Perform operations at the bit level.

- & (Bitwise AND), | (Bitwise OR), ^ (Bitwise XOR).
- << (Left Shift), >> (Right Shift).

- ~ (Bitwise Complement).

7. Ternary / Conditional Operator (?:) A shorthand for if-else.

- Syntax: Condition ? Value_if_True : Value_if_False

8. Special OOP Operators

- new: Allocates memory dynamically (on the heap).
- delete: Deallocates memory.
- :: (Scope Resolution Operator): Defines a function outside a class or accesses global variables.
- -> (Member Access Operator): Accesses class members using a pointer.
- . (Dot Operator): Accesses class members using an object instance.

Q2: Explain compile time and run time polymorphism in details.

Answer:

Polymorphism means "many forms." It allows a single function, operator, or object to behave differently in different contexts. It is divided into two main types:

1. Compile-Time Polymorphism (Static Binding)

This occurs when the compiler determines which function to call at compile time. It is faster execution-wise because the linkage happens early.

- **A. Function Overloading:** Multiple functions share the same name but have different parameter lists (different type or number of arguments).
    - *Example:* A function add(int a, int b) and add(double a, double b). The compiler decides which one to call based on the arguments passed.
- **B. Operator Overloading:** Giving special meaning to standard operators (like +, -) to work with user-defined data types (objects).
    - *Example:* Defining + to add two Complex number objects together.

2. Run-Time Polymorphism (Dynamic Binding)

This occurs when the exact function to be called is determined at runtime (while the program is executing). This is achieved using **Function Overriding** and **Virtual Functions**.

- **Virtual Functions:** A function declared with the virtual keyword in a base class. When a derived class overrides this function, the compiler uses a "v-table" (virtual table) to decide at runtime which function to call based on the type of object pointed to, not the type of the pointer.
- **Example Scenario:**
    - **Base Class:** Shape with a virtual function draw().
    - **Derived Classes:** Circle and Rectangle both override draw().
    - If you create a Shape* pointer and point it to a Circle object, calling ptr->draw() will execute the Circle's version of draw() at runtime. This is the essence of runtime polymorphism.

Q3: Define types of error and Explain exceptional handling in details.

Answer:

Part A: Types of Errors

Errors are mistakes or bugs that prevent a program from compiling or running correctly.

1. **Syntax Errors (Compile-time errors):** Violations of the programming language rules (e.g., missing semicolon, misspelled keywords). The compiler catches these.
2. **Runtime Errors:** Errors that occur during execution (e.g., dividing by zero, accessing an array out of bounds). These cause the program to crash.
3. **Logical Errors:** The program runs without crashing, but produces incorrect results due to flawed logic (e.g., using a wrong formula).
4. **Linker Errors:** Occur when the program compiles but the linker cannot find the definition of a function or library.

Part B: Exception Handling

Exception handling is a mechanism to handle runtime errors (exceptions) gracefully so that the normal flow of the application can be maintained, rather than crashing abruptly.

Key Keywords:

- **try:** A block of code where exceptions might occur. You place "risky" code here.
- **catch:** A block that catches and handles the exception thrown by the try block.

- **throw:** Used to manually generate an exception when a problem is detected.

How it works (The Flow):

1. The program executes code inside the try block.
2. If an error occurs (e.g., b == 0 in division), the program uses throw to send an exception object.
3. The control immediately exits the try block and searches for a matching catch block.
4. The catch block executes the error handling code (e.g., printing "Division by zero is not allowed").
5. The program continues execution after the catch block instead of terminating.

Example Structure:

C++

```
try {
  if (denominator == 0)
    throw "Division by zero error!";
  result = numerator / denominator;
}
catch (const char* msg) {
  cout << "Error Caught: " << msg << endl;
}
```

Q4: Write down short notes on:

(a) Text and Binary Stream

In C++ file handling, data is treated as a sequence of bytes, known as a **stream**.

1. Text Stream (Text Files):

- **Definition:** A stream of characters that humans can read.

- **Processing:** Special characters (like Newline \n) may undergo translation. For example, \n in memory might be converted to \r\n (Carriage Return + Line Feed) on disk in Windows.
- **Usage:** Good for storing documents, logs, and configuration files (e.g., .txt, .cpp).
- **Efficiency:** Slightly less efficient because of the translation/encoding overhead.

2. Binary Stream (Binary Files):

- **Definition:** A raw sequence of bytes exactly as they appear in memory.
- **Processing:** No translation happens. If the memory has a specific bit pattern, it is written to the file exactly that way.
- **Usage:** Used for images, audio, executables, or complex objects (serialization) where precision is required.
- **Efficiency:** More efficient and faster than text streams.

(b) File Updation and Creation

File handling involves creating, opening, reading, writing, and updating files using classes like ifstream (input), ofstream (output), and fstream (both).

1. File Creation (Opening): To work with a file, we use the open() function or the constructor. We must specify the **file mode**:

- ios::out: Opens file for writing (creates it if it doesn't exist).
- ios::in: Opens file for reading.
- ios::trunc: Deletes content if the file already exists (default for ios::out).

2. File Updation (Appending/Modifying): Updating a file means adding new data or changing existing data without deleting the old content.

- **Appending:** Use ios::app mode. This moves the file pointer (cursor) to the very end of the file, ensuring new data is added after the existing data.
- **Random Access Updation:** We can update specific parts of a file using file pointers:
    - seekp(position): Moves the "put" (write) pointer to a specific location.
    - seekg(position): Moves the "get" (read) pointer.

- This allows us to overwrite specific records in a binary file without rewriting the whole file.